02 · Métodos numéricos

De las ecuaciones matemáticas a algoritmos robustos, comprobables y aplicados a problemas reales de ingeniería.

MÓDULO	PUNTO DE PARTIDA	ENFOQUE
02_numerical_methods	Clase 17 · EX25	Ingeniería + Python + testing

Propósito del módulo

Aprender no solo a obtener un número, sino a justificar el algoritmo, estimar su error, verificar sus límites y explicar qué significa físicamente.

1 · Itinerario de aprendizaje

Seis bloques conectados, ordenados para desarrollar criterio matemático y habilidades de implementación. No hay fechas ni duración fija: avanzaremos cuando los conceptos y las validaciones estén consolidados.

01	**Álgebra numérica**	algebra/ Vectores, matrices, sistemas lineales y precisión numérica. Reparto de fuerzas y modelos de actuadores.
02	**Búsqueda de raíces**	root_finding/ Bisección, Newton-Raphson, tolerancias y convergencia. Resolver ecuaciones no lineales de ingeniería.
03	**Derivación numérica**	differentiation/ Diferencias finitas, pasos temporales y error numérico. Estimar velocidades y aceleraciones a partir de datos.
04	**Integración numérica**	integration/ Trapecios, Simpson, discretización y comparación. Acumular magnitudes físicas desde medidas o funciones.
05	**Estadística y probabilidad**	statistics/ Incertidumbre, propagación de errores y Monte Carlo. Caracterizar medidas y evaluar variabilidad.
06	**Ecuaciones diferenciales**	ode/ Euler, Runge-Kutta RK4, estabilidad y tamaño de paso. Simular sistemas dinámicos.

2 · Qué aprenderemos en cada bloque

Cada unidad combina conceptos, herramientas Python y una aplicación de ingeniería con comprobaciones reproducibles.

01 · ÁLGEBRA NUMÉRICA

Representar datos con arrays; operar con vectores y matrices; resolver A·x = b; interpretar residual, condicionamiento y precisión.

Python: NumPy · np.array · @ · np.linalg.solve · np.linalg.norm · np.linalg.cond

Engineering: EX25: asignación de fuerzas de dos actuadores. Validar fuerzas, residual y sensibilidad.

02 · BÚSQUEDA DE RAÍCES

Plantear f(x) = 0; diferenciar intervalo y estimación inicial; definir tolerancias; detectar no convergencia.

Python: Funciones · bucles · bisección · Newton-Raphson · criterios de parada

Engineering: Calcular una condición de equilibrio o un punto de operación de un sistema.

03 · DERIVACIÓN NUMÉRICA

Aproximar derivadas mediante diferencias hacia delante, atrás y centradas; comparar error y ruido.

Python: Arrays · slicing · np.diff · comparación con derivadas analíticas

Engineering: Extraer velocidad y aceleración a partir de muestras de posición.

04 · INTEGRACIÓN NUMÉRICA

Aproximar áreas e integrales con trapecios y Simpson; estudiar resolución y convergencia.

Python: Bucles · NumPy · acumulación · funciones de integración

Engineering: Estimar distancia, impulso o energía a partir de funciones o datos.

05 · ESTADÍSTICA Y PROBABILIDAD

Resumir muestras, cuantificar dispersión y propagación de incertidumbre; simular con Monte Carlo.

Python: NumPy · generador aleatorio reproducible · percentiles · gráficos

Engineering: Evaluar incertidumbres de sensores y propagarlas al resultado.

06 · ECUACIONES DIFERENCIALES

Discretizar ecuaciones dinámicas; implementar Euler y RK4; estudiar estabilidad y dependencia de Δt.

Python: Funciones de estado · integración temporal · arrays · matplotlib

Engineering: Simular un movimiento o sistema físico y contrastarlo con una referencia.

3 · Metodología de trabajo

Nuestro flujo estándar de resolución para todas las misiones del módulo.

01	**Problema físico**	Definir el sistema, unidades, hipótesis y pregunta de ingeniería.
02	**Matemáticas**	Formular ecuaciones, variables, condiciones y resultados esperados.
03	**Algoritmo**	Traducir el modelo a pasos implementables y criterios de parada.
04	**Python**	Escribir funciones claras, reutilizables y con entradas/salidas definidas.
05	**Validación**	Comparar resultados con soluciones conocidas o cálculos manuales.
06	**Interpretación**	Analizar errores, sensibilidad y sentido físico de los números.
07	**Testing**	Comprobar casos normales, límites y errores con tests automáticos.

4 · Materiales y entregables por clase

En cada clase o ejercicio: seis PDF (misión, guía de referencia y guía de teoría; cada uno en español e inglés). Al iniciar una sección, roadmap bilingüe específico.

5 · Organización del proyecto

```
Python_Engineering/
  02_numerical_methods/
    roadmap/
      Python_Engineering_Module_02_Roadmap_ES.pdf
      Python_Engineering_Module_02_Roadmap_EN.pdf
    algebra/  <- empezamos aquí
      class_17_vectors_and_linear_systems/
    root_finding/
    differentiation/
    integration/
    statistics/
    ode/
```

> **Criterio para dar por terminada una unidad**
>
> Entender el modelo y las funciones; resolver al menos una misión de ingeniería; verificar con resultados de referencia; explicar precisión y limitaciones; superar los tests pertinentes.

La numeración de clases y ejercicios continúa desde el módulo anterior. Clase 17 / EX25 es el primer ejercicio de este módulo; las siguientes misiones se asignarán a medida que avancemos.